VINAY.T

LINUX SCENARIO BASED Q&A DAY 9

Q1. Service Restarts Automatically Even After Being Disabled

🧩 Scenario:
You've disabled a service using systemctl disable, but it still starts automatically after reboot.

🧠 Possible Root Cause:
- The service is socket-activated by a .socket unit.
- A timer unit is triggering it on boot or schedule.

🛠️ Solution:
1. Check for socket activation:
   systemctl list-sockets | grep servicename

   If found, disable the corresponding socket unit:
   sudo systemctl disable servicename.socket

2. Check for timer-based activation:
   systemctl list-timers | grep servicename

   If found, disable the timer unit:
   sudo systemctl disable servicename.timer

✅ Tip:
Disabling a service doesn't automatically stop its socket or timer units; those need to be disabled separately to prevent auto-start.

Q2. System Time Incorrect Despite NTP Being Enabled

🧩 Scenario:
timedatectl status shows NTP as active, but the system clock is still incorrect.

🧠 Possible Root Cause:
- NTP server is unreachable
- Firewall is blocking NTP traffic (UDP port 123)

🛠️ Solution:
1. Check NTP sync status:
   chronyc sources
   or use ntpq -p if using ntpd.

2. Force NTP sync (if needed):
   sudo timedatectl set-ntp true
   sudo ntpdate pool.ntp.org

✅ Tip:
Ensure UDP port 123 is allowed through the firewall for accurate time synchronization.

Q3. systemctl Commands Are Extremely Slow

🧩 Scenario:
Running systemctl takes unusually long (10–20 seconds) to respond.

🧠 Possible Root Cause:
- Slow or failing DNS resolution
- systemd stalling while resolving hostnames or waiting on unresponsive systems

🛠️ Solution:
- Check and clean up slow DNS entries in /etc/hosts.
- If related to SSH, set UseDNS no in /etc/ssh/sshd_config.

✅ Tip:
Ensure hostnamectl status matches entries in /etc/hosts to avoid resolution delays.

Q4. Limit Resource Usage for a Service

🧩 Scenario:
You need to restrict CPU and memory usage for a specific service.

🧠 Possible Root Cause:
By default, systemd services can consume as many resources as the system allows. Without limits, a misbehaving or resource-intensive service can exhaust CPU or memory, degrading performance or causing instability.

🛠️ Solution:
1. Add the following directives to the service's unit file under the [Service] section:
   MemoryMax=500M
   CPUQuota=50%

2. Then reload the systemd daemon and restart the service:
   sudo systemctl daemon-reexec
   sudo systemctl restart <servicename>

✅ Tip:
Use systemd-cgtop to monitor real-time CPU and memory usage by systemd services.

Q5. Cronjob Runs but No Output is Captured

🧩 Scenario:
The cronjob executes as scheduled, but there's no output or log generated.

🧠 Possible Root Cause:
- Output isn't redirected, so it gets discarded by default.
- Script may fail silently without logging errors.

🛠️ Solution:
- Redirect both standard output and error to a log file:
  * * * * * /path/to/script.sh > /tmp/script.log 2>&1

✅ Tip: Always redirect both stdout and stderr in cronjobs to capture full execution details and aid debugging.

Q6. Cronjob Scheduled but Not Executing

🧩 Scenario:
A cronjob has been added to the crontab, but it doesn't run at the scheduled time. There's no output, and the expected task isn't performed.

🧠 Possible Root Cause:
- Incorrect cron syntax
- Missing or incorrect environment paths
- Script doesn't have execute permissions

🛠️ Solution:
1. Check cron syntax: Use crontab.guru or a validator to confirm correct scheduling.
2. Make script executable:
   chmod +x /path/to/script.sh

3. Use absolute paths:
   /usr/bin/python3 /home/user/scripts/myscript.py

4. Add environment variables if needed (e.g., PATH, HOME) at the top of the crontab:
   PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

5. Check cron logs:
- On most systems: grep CRON /var/log/syslog
- Or: journalctl -u cron

6. Verify cron daemon is running:
   systemctl status cron

✅ Tip:
Cron runs in a minimal shell with limited environment. Always use full paths, test your script manually, and log output like so:
* * * * * /path/to/script.sh > /tmp/script.log 2>&1

Q7. Cron Job Fails with "Permission Denied" Error

🧩 Scenario:
Cron attempts to execute a script but fails with a "Permission denied" message.

🧠 Possible Root Cause:
- The script lacks executable permissions (chmod +x not set).
- Ownership or directory permissions prevent access by the cron user.

🛠️ Solution:
1. Grant execute permission:
   chmod +x /path/to/script.sh
2. Ensure the script is owned by the correct user and accessible:
   chown user:user /path/to/script.sh
3. Check directory permissions leading to the script.

✅ Tip:
Cron runs with limited environment and user context — always verify script permissions and paths from the target user's perspective.

Q8. Not Receiving Email Alerts from Cron Jobs

🧩 Scenario:
Cron jobs encounter errors, but no email notifications are being received by the user.

🧠 Possible Root Cause:
Mail Transfer Agent (MTA) like sendmail or postfix is not installed or configured.
The system is unable to deliver emails from cron due to missing mail setup.

🛠️ Solution:

1. Install and configure an MTA (e.g., Postfix):
   sudo apt install postfix

2. Ensure the user running the cron job has a valid local mailbox or email routing setup.

3. Optionally, explicitly redirect cron output to a mail command:
   * * * * * /path/to/script.sh 2>&1 | mail -s "Cron Output" user@example.com

✅ Tip:
For systems without an MTA, use external SMTP tools like msmtp or redirect cron output to log files for review.

Q9. Run Cron Job Only on Weekdays

🧩 Scenario:
You want a script to run only from Monday to Friday.

🧠 Possible Root Cause:

- Incorrect weekday field in the cron expression
- Misunderstanding of cron's weekday numbering (0 or 7 = Sunday, 1 = Monday, 5 = Friday)
- System timezone mismatch affecting cron interpretation
- Cron service not restarted or reloaded after editing crontab

🛠️ Solution:

1. Schedule the cron as:
   0 9 * * 1-5 /path/to/script.sh

   This runs the script at 9:00 AM, Monday through Friday.

✅ Tip:
Always double-check with crontab -l and use date to verify the system time and day of the week. Use cron logs (e.g., /var/log/cron or journalctl -u cron) for troubleshooting.

Q10. Script Works Manually but Fails in Cron Due to Permissions

🧩 Scenario:
The script executes successfully when run from the terminal but fails with a permissions error when triggered by cron.

🧠 Possible Root Cause:
Cron runs in a different user context with limited environment variables and permissions. Files accessible in an interactive shell may not be accessible to cron.

🛠️ Solution:

- Ensure the script and any dependent files have appropriate read/write/execute permissions for the cron user.
- Check ownership: ls -l /path/to/script.sh
- Run: chmod +x /path/to/script.sh
- Test using: sudo -u <user> /path/to/script.sh

✅ Tip:

Always simulate cron environment by switching to the target user (su - username) before testing. Cron does not source full environment profiles like .bashrc.